`waitForSelector()` vs `locator.waitFor()`

Overview

Playwright provides two ways to explicitly wait for elements:

- `page.waitForSelector()` (Legacy approach)
- `locator.waitFor()` (Recommended approach)

`waitForSelector()`

``` ts const element = await page.waitForSelector('#btn', { state: 'visible' });

await element.click();

```
Characteristics

-   Selector-based
-   Returns an **ElementHandle**
-   Requires manual waiting
-   More flaky than locator-based approach

`locator.waitFor()`

``` ts
await page.locator('#btn')
  .waitFor({ state: 'visible' });

await page.locator('#btn').click();
```

Characteristics

- Locator-based
- No return value (continues working with the locator)
- Supports Playwright auto-waiting
- More stable and reliable

Comparison

`waitForSelector()` vs `locator.waitFor()` in Playwright

Feature	`page.waitForSelector()`	`locator.waitFor()`
Approach	Selector-based	Locator-based
Introduced	Earlier Playwright API (Legacy approach)	Modern Playwright Locator API (Recommended)
Syntax	`page.waitForSelector(selector)`	`locator.waitFor()`
Works With	CSS/XPath/Text selector	Locator object
Return Type	`Promise<ElementHandle>`	`Promise<void>` (No return value)
Element Reference	Returns an `ElementHandle`	Continues using the same `Locator`
Auto Waiting	Only waits for the specified state	Integrates with Playwright's built-in auto-waiting
Action Chaining	Not supported directly	Supported naturally with locator methods
Stability	Less stable	More stable
Dynamic Elements	More prone to failures	Handles dynamic DOM updates better
Readability	Requires extra variables	Cleaner and more concise code
Performance	Slightly less efficient due to `ElementHandle`	More optimized for Playwright
Recommended for New Code	❌ No	✅ Yes
Best Use Case	Legacy code or special `ElementHandle` operations	All modern Playwright automation

Advantages & Disadvantages

`waitForSelector()`	`locator.waitFor()`
✅ Returns `ElementHandle` when required	✅ Uses Locator API
✅ Useful for some low-level DOM operations	✅ Auto-waiting support
❌ Requires managing `ElementHandle`	✅ Cleaner code
❌ More prone to stale element issues	✅ Handles dynamic DOM updates
❌ Less maintainable	✅ Highly maintainable
❌ Considered a legacy approach	✅ Recommended by Playwright

Recommendation

Scenario	Recommended API
New Playwright projects	✅ `locator.waitFor()`
Existing legacy tests using `ElementHandle`	⚠️ `waitForSelector()` (only if necessary)
Dynamic web applications	✅ `locator.waitFor()`
Stable and maintainable automation framework	✅ `locator.waitFor()`
Following Playwright best practices	✅ `locator.waitFor()`

Best Practice

> ***Prefer `locator.waitFor()` and locator-based actions (`click()`, `fill()`, `check()`, etc.) over `page.waitForSelector()` for writing stable, reliable, and maintainable Playwright tests.***